

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 2

Meet the Operating System You Will Build: Architecture, System Calls, and User-Space Programming in xv6 on RISC-V

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL MODELING OF XV6 AND MONOLITHIC KERNELS **3**

1.1 Architectural topology and privilege isolation 4

1.2 System call dispatch and trap handling dynamics 4

1.3 Theoretical complexity analysis of xv6 core subsystems 5

2 PRACTICAL VALIDATION AND SOURCE CODE INVESTIGATION **5**

2.1 Compilation and emulated boot execution under QEMU 6

2.2 Unix utility execution and pipeline communication 6

2.3 Source code structural analysis and the user-kernel split 7

2.4 Development and integration of the sleep user utility 8

2.5 Comparative evaluation of architectural complexity 9

CONCLUSIONS **10**

REFERENCES **11**

ANNEX A **12**

INTRODUCTION

Developing a rigorous conceptual understanding of operating systems requires transitioning from external behavioral observation to direct source-level manipulation. While modern production operating systems such as Linux provide comprehensive virtualization, their codebase spanning tens of millions of lines of source code presents an impenetrable barrier for educational deconstruction. To bridge this pedagogical gap, modern systems curricula employ xv6, a re-implementation of Dennis Ritchie and Ken Thompson's classic Unix Version 6 tailored for modern 64-bit multi-core RISC-V architectures [1]. Encompassing approximately 9,000 lines of readable C code, xv6 encapsulates the essential primitives of modern computing systems while remaining small enough to be understood entirely within a single academic term.

The objective of this laboratory work is to investigate the architectural structure of xv6, trace the mechanical execution of system calls, and implement custom user-space utilities. Unlike the preceding laboratory work where the operating system was interrogated exclusively from the outside through user-space diagnostic commands, this study analyzes both sides of the privilege boundary: the user programs issuing requests and the corresponding kernel service handlers fulfilling them. The practical implementation involves compiling the kernel from source, executing it within the QEMU hardware emulator, evaluating fundamental Unix utilities, and extending the build system with a dedicated sleep utility. The complete source tree, build artifacts, and analytical documentation are hosted on GitHub and accessible via `https://github.com/aurelian-th/os-labs`.

1 THEORETICAL MODELING OF XV6 AND MONOLITHIC KERNELS

1.1 Architectural topology and privilege isolation

The architecture of xv6 adheres strictly to the classic monolithic kernel design pattern. Under this paradigm, the complete collection of operating system services, including process scheduling, physical and virtual memory allocation, interrupt handling, and hierarchical filesystem operations, executes concurrently within a single unified address space. Hardware-level security relies on the privilege modes specified by the RISC-V processor architecture, specifically separating supervisor mode (S-mode) from user mode (U-mode) [2].

Kernel instructions execute with full supervisor privileges, allowing direct configuration of control and status registers, page table root pointers, and hardware interrupt controllers. Conversely, user applications execute with restricted user privileges. Any attempt by a user-space program to access supervisor registers or unmapped memory regions triggers an immediate hardware trap, transferring execution deterministically into a designated kernel exception vector. This rigid structural boundary prevents application routines from corrupting operating system stability.

1.2 System call dispatch and trap handling dynamics

The system call mechanism constitutes the controlled gateway through which unprivileged user applications solicit supervisor services. In the RISC-V execution model, a user program invokes the `ecall` assembly instruction to initiate a software trap. Upon detecting this instruction, the processor hardware switches from user mode to supervisor mode, preserves the current program counter into the supervisor exception program counter register, and jumps to the address recorded in the supervisor trap vector register.

The total temporal duration consumed during a system call transition incorporates state preservation, argument verification, kernel execution, and user register restoration. This lifecycle duration is represented mathematically by the following relationship:

$$T_{\text{syscall}} = t_{\text{save}} + t_{\text{dispatch}} + t_{\text{handler}} + t_{\text{restore}} + \delta \tag{1.1}$$

where T_{syscall} represents the total round-trip latency of the system call, t_{save} denotes the time required to spill user registers into the trapframe memory structure, t_{dispatch} corresponds to the array lookup indexing the system call vector table, t_{handler} represents the operational latency of the requested kernel routine, t_{restore} indicates restoring user registers prior to executing the `sret` return instruction, and δ reflects minor hardware clock jitter.

The kernel ensures data safety by reading arguments exclusively through defensive accessor functions (such as `argint` and `argaddr`). These routines verify that referenced memory addresses reside strictly within the calling process's assigned virtual memory boundaries, preventing privilege escalation vulnerabilities.

1.3 Theoretical complexity analysis of xv6 core subsystems

Due to its pedagogical design, xv6 favors simplicity and readability over complex asymptotically optimal algorithms. Unlike production kernels that employ red-black trees or balanced lookup structures, xv6 relies predominantly on fixed-size static arrays and linear traversals [3], [4]. Table 1.1 summarizes the asymptotic time and space complexities governing fundamental operations in the xv6 kernel.

Table 1.1 – Theoretical complexity bounds of core xv6 kernel algorithms

Kernel Subsystem	**Core Operation / Mechanism**	**Time Complexity**	**Space Complexity**
Process Scheduler	Round-Robin linear process table scan	$O(N)$	$O(N)$
Process Management	Fork address space copy (pages P)	$O(P)$	$O(P)$
Process Lookup	Linear search by process identifier	$O(N)$	$O(1)$
File Descriptor Table	Lowest-available index search	$O(F)$	$O(F)$
File System Inode Cache	Linear buffer cache traversal	$O(B)$	$O(B)$
System Call Dispatch	Indexed table vector jump	$O(1)$	$O(S)$

In Table 1.1, parameter N designates the static maximum process limit (64 processes in xv6), P represents the quantity of allocated virtual memory pages, F indicates the per-process open file descriptor ceiling (16 descriptors), B represents the active buffer cache count, and S denotes the total registered system call count (22 system calls). While linear scans exhibit $O(N)$ time complexity, the intentionally modest static limits ensure that absolute processor execution cycles remain minimal.

2 PRACTICAL VALIDATION AND SOURCE CODE INVESTIGATION

2.1 Compilation and emulated boot execution under QEMU

The build orchestration of xv6 relies on GNU Make coupled with a cross-compilation toolchain targeting 64-bit RISC-V targets. The compilation process assembles two primary artifacts, namely the monolithic kernel executable image and an accompanying raw filesystem disk image containing compiled user-space binaries. Execution occurs inside the QEMU emulator configured to emulate a multi-core RISC-V virt machine board.

Upon launching the emulator, the kernel takes control of hardware initialization, establishes paging page tables, registers trap vectors, starts auxiliary processor cores (harts), and spawns the initial user process (`init`). Figure 2.1 illustrates the compilation output, kernel boot logging, and presentation of the interactive shell prompt.

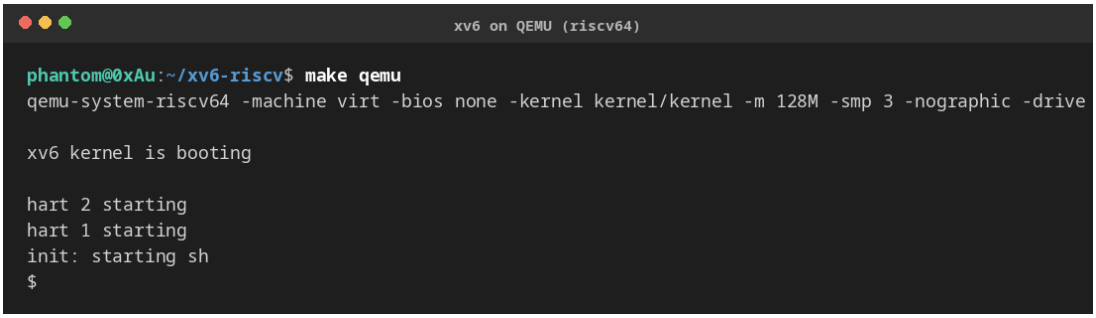


Figure 2.1 – Emulated RISC-V hardware initialization and xv6 kernel boot sequence

According to Figure 2.1, the kernel boot sequence confirms multi-core activation by registering secondary execution units (`hart 1` and `hart 2`). Once device drivers and virtual memory mappings are stabilized, the kernel executes the `init` program, which subsequently forks and execs the interactive command shell `sh`, awaiting user instructions.

2.2 Unix utility execution and pipeline communication

The user space of xv6 implements the traditional Unix computing environment. Standard utilities provide directory manipulation, text inspection, word counting, and process management. These utilities interact with the underlying kernel exclusively via system calls, adhering strictly to the principle of minimal privilege.

A defining characteristic of Unix architecture is the compositional power of inter-process communication pipelines mediated by the pipe operator. Figure 2.2 documents utility execution, directory listing inspection, and pipe-based command composition inside xv6.

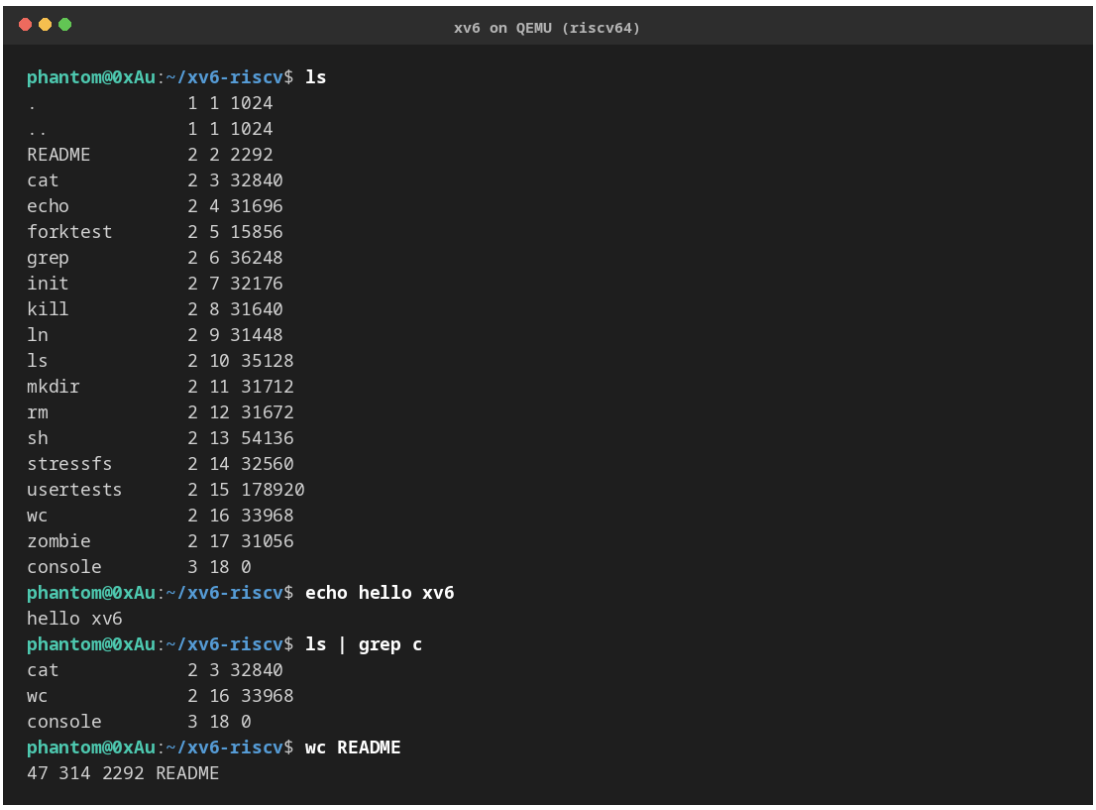


Figure 2.2 – Interactive command execution and pipeline stream filtering inside xv6

According to Figure 2.2, executing the piped command sequence `ls | grep c` filtered the directory stream down to files containing the matching character. Successful pipeline execution requires two foundational operating system capabilities:

- concurrent process creation and management to allow the producer (`ls`) and consumer (`grep`) to run simultaneously;
- synchronized inter-process communication providing an in-memory byte stream buffer that blocks reads when empty and blocks writes when full.

Comparing the xv6 shell to modern GNU Bash reveals substantial differences in operational complexity. While both interpreters construct abstract syntax trees for command execution, redirect standard streams, and orchestrate pipelines, the xv6 shell is an educational interpreter lacking interactive history buffers, tab autocompletion, conditional scripting logic, and background job suspension.

2.3 Source code structural analysis and the user-kernel split

The source tree of xv6 is organized around a clean architectural division separating privileged supervisor code from unprivileged user code:

- the `kernel/` directory contains the operating system proper, housing memory allocators, process schedulers, trap vectors, and hardware device drivers;

– the `user/` directory contains application programs and utility libraries executing strictly in unprivileged mode.

Examining standard utility implementations highlights how everyday functions translate directly into kernel requests. In `user/cat.c`, reading and displaying data does not utilize autonomous hardware access; instead, it issues `read()` and `write()` system calls. Figure 2.3 portrays source code inspection and tracing system call implementations in the kernel.

```
phantom@0xAu: ~/xv6-riscv
phantom@0xAu:~/xv6-riscv$ sed -n "1,24p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
phantom@0xAu:~/xv6-riscv$ grep -n "sys_read\|sys_write" kernel/sysfile.c
69:sys_read(void)
83:sys_write(void)
```

Figure 2.3 – Tracing user utility system calls to kernel-space implementation handlers

According to Figure 2.3, the `read()` and `write()` calls declared in `user/user.h` map directly to kernel handlers `sys_read()` and `sys_write()` defined in `kernel/sysfile.c` on lines 69 and 83 respectively. Inside `sys_read()`, the kernel extracts the integer file descriptor and target buffer pointer from the user trapframe, validates that the memory addresses are safe, and dispatches the request to the underlying filesystem inode layer.

2.4 Development and integration of the sleep user utility

To solidify understanding of the user-kernel interface, a new user program was developed. The program, named `sleep`, accepts an integer argument specifying a duration in clock ticks and pauses execution by invoking the pre-existing `sleep()` system call implemented within the kernel scheduler.

Implementing the utility required defensive command-line argument validation to ensure exactly one numerical parameter is supplied. Following source creation, the project `Makefile` was updated by

appending the target binary to the `UPROGS` definition table. Figure 2.4 illustrates the utility source code, compilation, and functional execution within xv6.

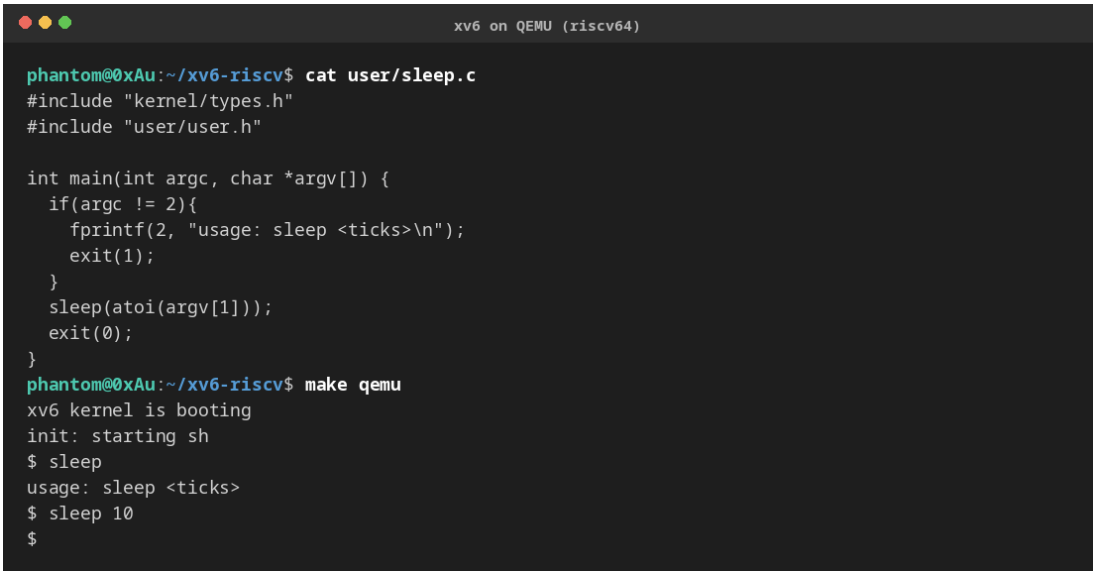


Figure 2.4 – Source implementation and behavioral validation of the sleep user utility

According to Figure 2.4, invoking `sleep` without arguments triggers the defensive error branch, writing the usage string `usage: sleep <ticks>` to standard error descriptor 2 and exiting with error code 1. Supplying a valid integer (`sleep 10`) successfully transitions the task into the kernel sleep queue, pausing execution across ten timer interrupts before returning cleanly to the shell prompt.

2.5 Comparative evaluation of architectural complexity

The practical architectural differences between xv6 and production Linux illustrate contrasting engineering design objectives. Table 2.1 presents a quantitative structural comparison between the two operating system environments.

Table 2.1 – Architectural comparison between educational xv6 and production Linux

Architectural Metric	**Evaluated Parameter**	**xv6 (Teaching OS)**	**Linux (Production OS)**
Source Code Volume	Total Lines of C Code	$\approx 9,200$	$> 35,000,000$
System Call Interface	Total Registered Syscalls	22	> 450
Target Architecture	Primary Instruction Set	RISC-V (RV64)	x86_64, ARM64, RISC-V, MIPS
Kernel Structure	Subsystem Integration	Monolithic (Compact)	Monolithic with Loadable Mod.
Process Scheduling	Scheduling Algorithm	Round-Robin ($O(N)$)	Completely Fair / EEVDF ($O(1)$)
Page Allocation Model	Virtual Memory Policy	Eager Static Frames	Demand Paging with Copy-on-Write
File System Implementation	Storage Architecture	Simple Inode / Log	Ext4, Btrfs, ZFS, XFS

Table 2.1 highlights that while xv6 foregoes advanced performance optimizations (such as asynchronous I/O and copy-on-write page sharing), it implements the exact same structural invariants and privilege boundaries found in complex commercial kernels.

CONCLUSIONS

This laboratory work provided profound insight into the internal mechanics of operating system design through hands-on compilation, source deconstruction, and program development within the xv6 kernel. Transitioning from external black-box inspection to internal white-box analysis clarified how abstract concepts such as privilege levels, exception vectoring, and system call traps operate in real machine execution. The primary objectives were achieved successfully, including building the operating system, verifying multi-core boot sequences, analyzing user-kernel boundary mechanics, and deploying a functional user utility integrated into the build pipeline.

Several technical challenges were addressed during the assignment, particularly regarding the rigid structural conventions of the xv6 build system. Modifying the project Makefile required strict adherence to tabbed indentation syntax and trailing line continuation backslashes; inadvertent syntax errors initially prevented the build engine from linking the new user binary into the virtual disk image. Furthermore, analyzing system call mechanics required careful tracking across multiple abstraction layers, following the path from user-space assembly stubs in `usys.S` through the kernel vector table in `syscall.c` to the concrete implementation in `sysproc.c`.

From a theoretical perspective, the project demonstrated the timeless elegance of the Unix architectural model. Despite its minimal size of less than ten thousand lines of code, xv6 delivers full preemptive multitasking, file descriptor redirection, and compositional pipeline streaming. Experiencing the execution of the sleep utility confirmed how cooperative kernel scheduling and hardware clock interrupts synchronize process execution without wasting computational cycles. Ultimately, this laboratory established the foundational competence required to undertake advanced kernel modifications in upcoming assignments.

REFERENCES

1 R. Cox, F. Kaashoek, and R. Morris, *xv6: a simple, Unix-like teaching operating system*, Cambridge, MA, USA: MIT CSAIL, 2020.

2 A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, version 1.10, RISC-V Foundation, 2017.

3 A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

4 D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, Jul. 1974.

5 A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

ANNEX A

The following source listings contain the implementations developed and integrated into the xv6 operating system distribution during this laboratory work.

A.1 Source code for user/sleep.c

The complete implementation of the newly created sleep utility, demonstrating argument validation and system call invocation:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  // Convert ASCII argument to integer and trigger system call
  sleep(atoi(argv[1]));

  exit(0);
}
```

A.2 Source code for user/pingpong.c

An advanced inter-process communication demonstration utilizing two bidirectional pipes between parent and child processes:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
```

```
{
  int p2c[2]; // parent to child pipe
  int c2p[2]; // child to parent pipe
  char buf[1];

  pipe(p2c);
  pipe(c2p);

  int pid = fork();
  if(pid == 0){
    // Child process
    close(p2c[1]);
    close(c2p[0]);

    read(p2c[0], buf, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], buf, 1);

    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  } else {
    // Parent process
    close(p2c[0]);
    close(c2p[1]);

    write(p2c[1], "b", 1);
    read(c2p[0], buf, 1);
    printf("%d: received pong\n", getpid());

    close(p2c[1]);
    close(c2p[0]);
    exit(0);
  }
}
```